

JAVA PROGRAMMING MASTERCLASS

抽象類別

「定義骨架，交由子類別實作」

[← 返回目錄](#)

Outline

- 抽象類別 (**Abstract Class**)
- 抽象方法 (**Abstract Method**)
- 觀念整理
- 進階應用 — 建構方法、Upcasting、Sealed Classes、Template Method Pattern
- 抽象類別 **vs** 介面
- 實作練習

抽象類別

Abstract Class

什麼是抽象類別

- 使用關鍵字 **abstract** 宣告的類別稱為**抽象類別**
- 抽象觀念主要是**隱藏工作細節**，使用者只需知道如何使用
 - 例如 **+** 符號可以執行數值加法，也可以執行字串相加
 - 但不需要知道內部程式如何設計 **+** 號的功能
- 這個類別中可以有**抽象方法**（ abstract method ）和**實體方法**（ method ）

使用抽象類別の場合

- 有一個 **Shape** 類別包含計算繪製外型的 **draw()** 方法
 - **Circle** 和 **Rectangle** 繼承 **Shape**，各自重新定義外型繪製
- **Shape** 的存在讓整個程式定義更加完整，本身不處理任何工作
 - 真正的工作交由子類別完成
 - 這就是一個適合使用**抽象類別** (abstract class) の場合

💡 **核心概念：** 抽象類別是「模板」，子類別依照各自情境對此模板擴展和建構。

抽象類別語法

- 在定義類別名稱的 `class` 左邊加上 `abstract` 關鍵字

```
1  abstract class Shape {  
2      // 類別內容  
3  }
```

- 抽象類別定義的方法（實際執行的部分）交由子類別重新定義
- 可以把抽象類別想成是一個**模板**，子類別依照各自情境擴展和建構

抽象類別不能實例化

- 抽象類別不能使用 `new` 建立物件（不能實例化）
- 繼承（實作）的子類別可以實例化

```
1  abstract class Shape {  
2      public void draw() { }  
3  }  
4  // Shape shape = new Shape(); // 編譯錯誤！  
5  Circle circle = new Circle(); // 子類別可以
```

💡 錯誤訊息：'Shape' is abstract; cannot be instantiated

抽象類別實作 – 骨架定義

- 繪製框架 (**Shape**) : 抽象類別，定義 **draw()** 方法骨架
- 各自實作 (**Circle** 、 **Square**) : 子類別各自 override **draw()**

```
1  abstract class Shape {  
2      public void draw() { }           // 純定義，無實作內容  
3  }  
4  class Circle extends Shape {  
5      @Override  
6      public void draw() {  
7          System.out.println("繪製圓形!");  
8      }  
9  }
```


抽象類別實作 – 子類別範例

```
1  class Square extends Shape {  
2      @Override  
3      public void draw() {  
4          System.out.println("繪製矩形!");  
5      }  
6  }  
7  // 使用方式：  
8  // Circle cir = new Circle();  cir.draw();  → 繪製圓形！  
9  // Square squ = new Square();  squ.draw();  → 繪製矩形！
```

抽象方法

Abstract Method

抽象方法的特性

特性	說明
沒有實體內容	無方法主體 (no body)
宣告以 <code>;</code> 結尾	不使用 <code>{ }</code> 大括號
必須被子類別 <code>override</code>	子類別 必須 重新定義，否則編譯錯誤
類別需宣告為 <code>abstract</code>	含抽象方法的類別必須是抽象類別

抽象方法的特性 – 範例

```
1  abstract class Shape {  
2      public abstract void draw(); // 抽象方法 (以 ; 結尾)  
3  }  
4  class Circle extends Shape {  
5      @Override  
6      public void draw() {           // 子類別重新定義  
7          System.out.println("繪製圓形!");  
8      }  
9  }
```

💡 注意：子類別重新定義時，回傳值型態與參數個數、型態需與抽象方法一致，並建議加上 **@Override**。

子類別未重新定義抽象方法

- 若子類別沒有重新定義抽象方法，編譯時會出現錯誤
- 解法：將子類別也宣告為抽象類別，延遲到孫類別實作

```
1  abstract class Car {  
2      abstract void run();    // 抽象方法  
3  }  
4  class Bmw extends Car {  
5      // 未 override run() → 編譯錯誤！  
6      // Class 'Bmw' must implement abstract method 'run()' in 'Car'  
7  }
```

觀念整理

Abstract Class & Method

抽象類別與抽象方法 – 重要規則

規則	說明
抽象類別無法實例化	必須透過子類別建立物件
含抽象方法 → 必須宣告為 abstract class	普通類別中不存在抽象方法
子類別必須 Override 所有抽象方法	否則子類別也必須宣告為 abstract
抽象類別不一定要有抽象方法	可以只包含普通方法
抽象類別可以混用兩種方法	抽象方法 + 普通方法皆可存在

抽象類別可以有兩種方法

- 抽象類別內可以同時有抽象方法和普通方法

```
1  abstract class Car {  
2      abstract void run();    // 抽象方法  
3      void refuel() {         // 普通方法  
4          System.out.println("汽車加油");  
5      }  
6  }
```

抽象類別兩種方法 – 子類別使用

```
1  class Bmw extends Car {  
2      @Override  
3      public void run() {  
4          System.out.println("安全駕駛中 ... ");  
5      }  
6  }  
7  // Bmw bmw = new Bmw();  
8  // bmw.refuel();    → 汽車加油  
9  // bmw.run();       → 安全駕駛中 ...
```


進階應用

Constructor & Upcasting

抽象類別的建構方法

- 設計 Java 程式時，可以將**建構方法**（ constructor ）或**屬性**（ 成員變數 ）的觀念應用在抽象類別
- 子類別建立物件時，會先執行父類別（ 抽象類別 ）的建構方法

```
1  abstract class Car {  
2      abstract void run();  
3      Car() {  
4          System.out.println("有車子了");  
5      }  
6      void refuel() {  
7          System.out.println("汽車加油");  
8      }  
9  }
```


抽象類別的建構方法 – 範例

```
1  class Bmw extends Car {  
2      public void run() {  
3          System.out.println("安全駕駛中 ... ");  
4      }  
5  }  
6  public static void main(String[] args) {  
7      Bmw bmw = new Bmw(); // 建立物件時先執行 Car()  
8      bmw.refuel();  
9      bmw.run();  
10 }  
11 // 輸出順序：有車子了 → 汽車加油 → 安全駕駛中 ...
```

抽象類別的屬性宣告

概念

說明

protected 屬性

子類別可直接存取，外部無法存取

super(參數)

子類別透過 **super()** 初始化父類別屬性

```
1  abstract class Car {  
2      protected String brand;  
3      Car(String brand) { this.brand = brand; }  
4      abstract void run();  
5  }
```


抽象類別屬性 – 子類別使用範例

```
1  class Bmw extends Car {  
2      Bmw() { super("BMW"); }  
3      @Override  
4      public void run() {  
5          System.out.println(brand + " 行駛中");  
6      }  
7  }  
8  Car bmw = new Bmw();    // Upcasting  
9  bmw.run();              // BMW 行駛中
```

💡 子類別直接使用 **brand**，無需重複宣告，因為 **protected** 屬性可由子類別繼承

使用 Upcasting 宣告抽象類別的物件

- 抽象類別無法實例化，但可以用 **Upcasting**（向上轉型）宣告物件
- 用父類別型態接住子類別 **new** 出來的物件

```
1  Car bmw = new Bmw();    // Upcasting (合法)
2  // Car car = new Car(); // 編譯錯誤！
3  bmw.refuel();
4  bmw.run();
```

💡 常見用法：許多 Java 程式設計師會使用 Upcasting 宣告抽象類別物件，由所宣告物件的參考是子類別，所以可以正常執行工作。

密封抽象類別 (Sealed Abstract Class)

Java 17 起，`sealed` 可搭配 `abstract` 一起使用，精確限制哪些類別可以繼承該抽象類別：

關鍵字	說明
<code>sealed</code>	宣告該類別為密封類別
<code>permits</code>	指定允許繼承的子類別清單

```
1 // 限制只有 Circle 和 Square 可以繼承 Shape
2 public abstract sealed class Shape permits Circle, Square {
3     public abstract double area();
4 }
```


密封子類別的修飾詞

繼承密封類別的子類別，必須使用以下修飾詞之一：

修飾詞	說明
final	終止繼承，不能再有子類別
sealed	繼續密封，需指定新的 permits
non-sealed	解除限制，任何類別皆可繼承

```
1 public final class Circle extends Shape { /* ... */ }
2 public non-sealed class Square extends Shape { /* ... */ }
```

Template Method Pattern

抽象類別的經典設計模式：用 **final** 方法固定流程骨架，用 **abstract** 方法讓子類別填入細節：

方法角色	宣告方式	說明
骨架方法	final 普通方法	定義固定流程，子類別不可 Override
可變步驟	abstract 方法	子類別各自實作細節

```
1  abstract class Game {  
2      abstract void start();    // 可變步驟  
3      abstract void end();      // 可變步驟  
4      final void play() { start(); end(); } // 骨架固定  
5  }
```


Template Method Pattern – 子類別實作

```
1  class Chess extends Game {  
2      @Override void start() { System.out.println("走棋"); }  
3      @Override void end()   { System.out.println("將軍"); }  
4  }  
5  class Soccer extends Game {  
6      @Override void start() { System.out.println("踢球"); }  
7      @Override void end()   { System.out.println("進球"); }  
8  }
```



new Chess().play() → 走棋 → 將軍；新增遊戲只需新增子類別，骨架流程不需修改

抽象類別 vs 介面

Abstract Class vs Interface

抽象類別與介面的比較

比較項目	抽象類別 Abstract Class	介面 Interface
父類別/父介面繼承	只能繼承一個類別	能繼承多個介面 (Java 實現多重繼承)
子類別繼承/實作	extends 一個抽象類別	implements 多個介面
方法	可包含非抽象方法	只能是抽象方法 (Java 8 以前)
必定為	父類別	可視為抽象類別的特例

介面的演進 (Java 8+)

Java 8 起，介面支援 **default** 與 **static** 方法（Java 9 加入 **private**），與抽象類別的差異縮小。但介面仍無法儲存狀態（無實例欄位），需要共用屬性時仍應使用抽象類別。

💡 介面 **default**、**static**、**private** 方法的詳細用法，將在 Ch17 介紹。

抽象類別與介面 – 相同點與應用

相同點：

- 兩者都無法直接實體化
- 子類別都必須實作已宣告之抽象方法（或繼續抽象）

應用場景比較：

- 抽象類別：關係密切的類別中，如定義抽象類別 **Car**，子類別 **Benz** 及 **Audi** 繼承 **Car**
- 介面：定義一些功能給不相干類別使用，如定義介面 **Fly**，子類別 **AirPlane** 及 **Bird** 實作 **Fly**

實作練習

練習 1：Shape 面積與周長

任務說明

請設計一個抽象類別 **Shape**，包含計算面積 (**area()**) 和周長 (**perimeter()**) 的抽象方法，再設計 **Rectangle** 和 **Circle** 兩個子類別分別實作。

預期輸出：

```
1  矩形面積：6.0
2  矩形周長：10.0
3  圓面積：12.566370614359172
4  圓周長：12.566370614359172
```


練習 1：解題提示

提示說明

1. 在 `Shape` 中宣告 `abstract double area();` 與 `abstract double perimeter();`
2. `Rectangle` 需 `height`、`width` 屬性，透過建構方法傳入 (高 2，寬 3)
3. `Circle` 需 `r` (半徑) 屬性，透過建構方法傳入 (半徑 2)
4. 計算公式：
 - 矩形面積 = `height * width`，周長 = `2 * (height + width)`
 - 圓面積 = `Math.PI * r * r`，圓周長 = `2 * Math.PI * r`

練習 2：抽象數學計算器

任務說明

請設計一個抽象類別 `MyMath`，包含 `add()` 與 `mul()` 兩個帶參數的抽象方法，以及一個普通方法 `output()` 印出「我的計算器」。設計子類別 `MyTest` 重新定義這兩個抽象方法。

預期輸出：

```
1  我的計算器
2  加法結果：11
3  乘法結果：24
```


練習 2：解題提示

提示說明

1. 在 `MyMath` 中宣告 `abstract int add(int n1, int n2);` 與 `abstract int mul(int n1, int n2);`
2. 普通方法 `void output()` 直接印出「我的計算器」，不需 override
3. 在 `MyTest` 中實作：`add()` 回傳 `n1 + n2`，`mul()` 回傳 `n1 * n2`
4. 在 `main` 中使用 Upcasting：`MyMath obj = new MyTest();`
5. 呼叫 `obj.output()`、`obj.add(3, 8)`、`obj.mul(3, 8)`

Q & A

實戰加料

為期中作業暖身 🔥

實戰：用抽象類別設計問卷題目

問卷有三種題型：單選、多選、簡答。每種題目都有「標題」和「必填」，但驗證答案的方式各不同——抽象類別是最自然的設計：

```
1  abstract class AbstractQuestion {
2      protected String title;    // 題目文字
3      protected boolean required; // 是否必填
4
5      AbstractQuestion(String title, boolean required) {
6          this.title = title;
7          this.required = required;
8      }
9
10     // 各題型自行決定怎麼驗證
11     public abstract boolean validate(String answer);
12
13     // 共用：必填且答案空白就不合法
14     public boolean isBlankWhenRequired(String answer) {
15         return required && (answer == null || answer.isBlank());
16     }
17 }
```


實戰：三種題型子類別

```
1  // 單選題：答案必須是選項之一
2  class SingleChoice extends AbstractQuestion {
3      String[] options;
4      SingleChoice(String title, boolean required, String... options) {
5          super(title, required); this.options = options;
6      }
7      @Override
8      public boolean validate(String answer) {
9          if (isBlankWhenRequired(answer)) return false;
10         for (String opt : options) if (opt.equals(answer)) return true;
11         return false;
12     }
13 }
14
15 // 多選題：分號分隔，每個答案都必須是合法選項
16 class MultiChoice extends AbstractQuestion {
17     String[] options;
18     MultiChoice(String title, boolean required, String... options) {
19         super(title, required); this.options = options;
20     }
21     @Override
22     public boolean validate(String answer) {
23         if (isBlankWhenRequired(answer)) return false;
```

課程結束

感謝聆聽，有問題請發問！